Faculty of Sciences Departement of Computer Science

PREPARATORY WORK FOR THE MASTER THESIS

Using LLMs for Multi-Agent Collaboration in the Laser Learning Environment

Author : Sofia Chica Cobo

Supervisor : Yannick Molinghen

Director : Tom Lenaerts

Academic year 2024–2025

Contents

1 Introduction **1**

2 State of Art **2**
2.1 The Laser Learning Environment (LLE) 2
2.1.1 Existing RL Strategy for Multi-Agent Collaboration in LLE task 3
2.2 Large Language Models for Multi-Agent Collaboration 5
2.2.1 Background Concepts 5
2.2.2 LLM-based agents for Collaborative Environments 6
2.2.3 Other LLM-Based Multi-Agent Frameworks 8

3 Proposed Approach **10**
3.1 Using LLM in the LLE 10
3.1.1 Exploring multi-agent collaboration with LLM 10
3.1.2 Exploring multi-agent collaboration with LLM and VDN . 14
3.2 Technical aspect of the implementation 15
3.2.1 About the LLMs models 15
3.2.2 Integrating LLM-based Agents in the LLE Environment . 17
3.2.3 Results Evaluation 17

4 Conclusion **19**

Bibliography **20**

Chapter 1

Introduction

Multi-agent reinforcement learning (MARL) focuses on training multiple agents to work together in shared environments. Each agent explores, makes decisions, and updates its behavior based on rewards. In cooperative setups, the challenge comes from the fact that the agents need to work together to achieve their mission. The Laser Learning Environment (LLE) is one such case, where agents must cooperate to overcome obstacles like laser beams, unreachable zones, and shared bottlenecks [6].

Traditional MARL methods, such as value-based deep Q-networks, have achieved promising results on the different levels of LLE. However, they still struggle on the most difficult tasks, especially when long-term planning and unrewarded cooperation are required.

This project explores whether large language models (LLMs) can support multi-agent collaboration in LLE. Rather than learning from scratch, LLM agents rely on structured prompts, memory representations, and theory of mind (ToM) reasoning to plan and collaborate.

The main objectives are:

- Introduce LLM-based agents in the Laser Learning Environment;
- Enable agents to communicate and make cooperative decisions using LLM reasoning;
- Explore how belief state prompting and ToM can support coordination;
- Measure the impact on score, exit rate, and qualitative teamwork behaviors.

Chapter 2

State of Art

2.1 The Laser Learning Environment (LLE)

The Laser Learning Environment is a multi-agent grid world populated by agents of different colours. It includes various types of tiles: floor, start, wall, laser, laser source, and exit tiles. The main objective in this environment is for all agents to reach the exit tile.

However, to do so, the agents must overcome several constraints. Since laser beams are deadly to agents of a different colour, an agent cannot cross a laser unless it matches their colour. To get around this, agents must cooperate, one can block a laser beam, allowing another to pass safely. This makes the game fully cooperative, as success depends on helping teammates advance through the level.

The points distribution is as follows:

- +1 point per agent that reaches the exit tile;
- +1 point if all agents reach the exit tile;
- 0 points for intermediate cooperative actions;
- Negative outcome: agent death.

Additionally, hidden gems scattered throughout the map provide bonus points when collected by agents.

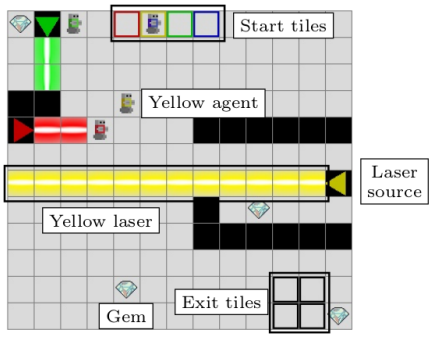

Figure 2.1: Level 6 of the Laser Learning Environment.

This environment is particularly relevant because it introduces key coordination challenges:

- **a. Perfect Coordination**
  Agents must follow a precise sequence of actions. A single mistake, like moving too early or too late, can lead to failure. For example, if an agent does not block a laser at the right moment, a teammate may be eliminated.

- **b. Interdependence**
  Agents depend on one another to complete the task. Some areas of the map are inaccessible to a single agent acting alone. Only by helping each other they can progress.

- **c. Zero-Incentive Dynamics**
  Essential cooperative actions, like blocking a laser for another agent, provide no direct reward. This forces agents to learn helpful behaviours even in the absence of positive reward.

The most interesting level in LLE is the level 6 shown in the Figure 2.1 which has the highest level of interdependence. It features 4 agents, 3 lasers, and 4 hidden gems.

2.1.1 Existing RL Strategy for Multi-Agent Collaboration in LLE task

To address cooperative multi-agent environments challenges like the Laser Learning Environment (LLE), researchers have explored various reinforcement learning (RL) algorithms. Similar cooperative problems have been studied in environments such as Hanabi [1] and Overcooked [15].

In these contexts, Q-learning-based methods have been used a lot to train agents, in this Q-learning setup agents learn action policies by maximizing expected rewards over time.

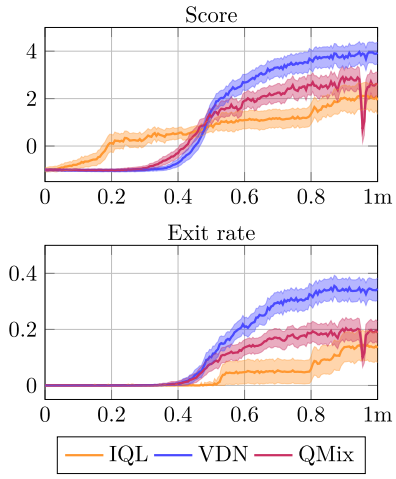

Figure 2.2: Training score and exit rate over time for IQL, VDN and QMIX on level 6 2.1. [6]

However, when multiple agents are involved, things get more complicated. Each agent influences the environment, so what one agent learns changes the experience of the others. This problem is known as **non-stationarity**, and it makes learning stable strategies much harder.

Value Decomposition Networks (VDN)

One way to deal with the non-stationarity problem in cooperative multi-agent environments is the Value Decomposition Network (VDN) [13]. VDN introduces a simple yet effective idea: rather than learning a single joint Q-function for the entire team, the total team value is approximated as the sum of the individual Q-values of each agent.

Formally:

$$Q_{total}(s, a_1, ..., a_n) \approx \sum Q_i(s_i, a_i)$$

VDN addresses the non-stationarity issue and enables scalable learning in cooperative tasks.

According to the results in Figure 2.2 from the original LLE paper [6], VDN was the most successful among the tested baseline algorithms. While more advanced methods like QMIX [10] were also evaluated, VDN performed better in terms of both score and exit rate, especially on Level 6, which has the highest level of interdependence among all the levels.

For this reason, VDN serves as a solid baseline to see if LLM-based agents can come close to or match the kind of performance learning-based methods achieve.

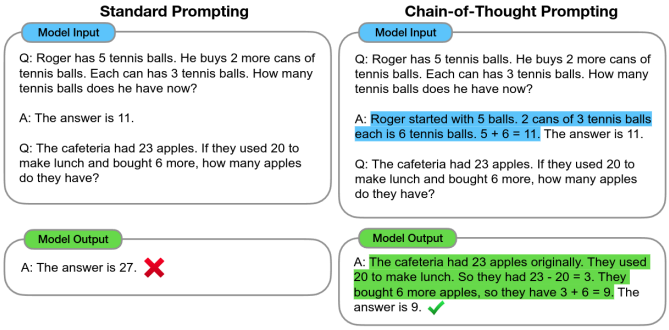

Figure 2.3: Chain-of-thought prompting usage [14]

2.2 Large Language Models for Multi-Agent Collaboration

2.2.1 Background Concepts

Theory of Mind (ToM)

The Theory of Mind refers to the cognitive ability to understand and reason about the beliefs, intentions, and knowledge of others. In psychology, it is considered a core aspect of social intelligence, necessary for effective communication, empathy, etc.

For artificial agents, incorporating ToM involves the ability to infer not just what is true in the environment, but what other agents know or believe. For example, an agent might reason: "Agent B hasn't seen the bomb yet" or "Agent A thinks I'm unaware of the location." These kinds of inferences require the agent to model the mental states of others.

Chain of thought (CoT) Prompting

Chain-of-Thought (CoT) prompting is a method that encourages language models to generate intermediate reasoning steps before arriving at a final answer. Rather than immediately producing a solution, the model is prompted to reason through the task step-by-step in natural language. This technique has been shown to significantly improve performance on complex tasks such as mathematical reasoning or logic puzzles see Figure 2.3.

In multi-agent settings, CoT prompting can be used not only to improve an individual agent's reasoning [14] but also to facilitate clearer communication between agents [5].

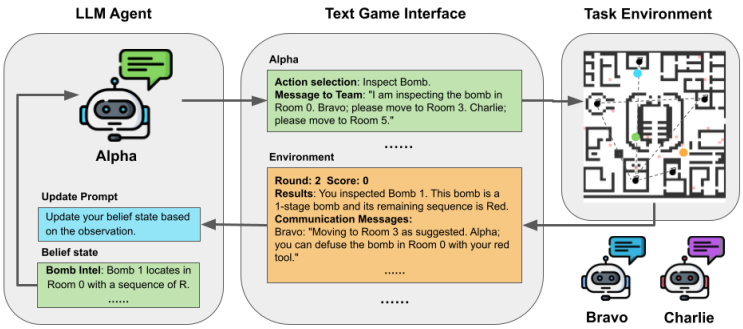

Figure 2.4: Proposed framework [5] that consists of 3 LLM-based agents, a text game interface and the actual task environment. The natural language outputs of LLM are encoded into abstract actions by the text interface and then sent to task environment. The task environment then processes agent actions and returns observations via the text interface. Upon receiving environmental observations, LLM-based agents are prompted to update their beliefs and output action selections and messages

2.2.2 LLM-based agents for Collaborative Environments

The LLM-Based Agent

Large Language Models (LLMs), particularly GPT-4, have recently been used as agents for solving multi-agent collaboration tasks. Unlike traditional reinforcement learning (RL) approaches, which rely on repeated interactions and a system of rewards to train agent behavior, LLM-based agents function differently. They are not trained within the environment. Instead, they rely on zero-shot or few-shot prompting, using only structured instructions, task descriptions, and their pre-trained internal logic to make decisions.

A recent study by `Li et al.`[5] presents a text-based collaborative task where three LLM-based agents need to work together to defuse color-coded bombs scattered in an unexplored environment 2.4. The key difficulty lies in respecting the correct sequence of actions needed to defuse each bomb while not having a full knowledge of the environment. LLM-based agents are not told exactly what to do; they rely only on the LLM logic and the mission context given. Despite the absence of explicit multi-agent training, the LLM-based agents demonstrated emergent cooperative behaviors such as coordinated task allocation, synchronized movement and strategy negotiation and delegation.

Theory of Mind Reasoning in LLM-based Agents

The ToM reasoning in LLM-based agents can be tested through their capacity to interpret partial observations, estimate what other agents may or may not know, and adapt their behavior accordingly.

The ToM capabilities in LLM-based agents are evaluate through a structures framework of reasoning levels. These include [5]:

- **Introspection:** the agent's ability to describe its own beliefs or mental state.
- **First-order ToM:** reasoning about what one agent believes about other's mental state (e.g., "I believe that Agent B hasn't seen that the bomb has been defused").
- **Second-order ToM:** reasoning about what one agent believes about their own mental state (e.g., "Agent A believes that I don't know Room 5 has been cleared").

This ToM capability is essential in decentralized setups where agents have limited views and and need to guess what others know in order to work together.

Belief State Prompting as Externalized Memory

One limitations of LLMs in multi-step environments is their lack of persistent memory. They can easily lose track of what has been observed or communicated previously.

To overcome this, Li et al. [5] introduce a technique known as **belief state prompting** inspire by the chain-of-thought prompting [14]. Due to input length constraints, the full interaction history cannot be included at every step. Instead, each agent maintains a structured, compressed summary of its understanding of the world, encoded as a belief state and injected into the prompt at each round.

This belief state includes:

- A summary of recent observations;
- Inferred knowledge or intentions of teammates

This design helps the agent reason coherently across steps. It acts as an externalized memory system that simulates state tracking and also allows for ToM-based reasoning.

Adding belief state prompting helps LLM agents reason better about what others know or believe, making Theory of Mind reasoning more accurate.

Performance and Limitations

When equipped with belief state prompting, LLM-based agents in the bomb defusal task consistently reached high levels of performance[5]. In most runs, they were able to complete all bomb sequences with near-perfect scores.

Nevertheless, some difficulties remain. Some limitations include the generation of invalid actions (e.g., trying to defuse non-existent bombs) or hallucinations about the task state (e.g., assuming tools are available when they are not). These errors often stem from inconsistencies in the belief state or the natural limits of the model's context length.

2.2.3 Other LLM-Based Multi-Agent Frameworks

Beyond the bomb defusal task, several other frameworks have been proposed to explore how large language models can support multi-agent collaboration across different domains. Below, we briefly present three examples.

LLM-Collab

A framework designed to enhance task planning by combining the reasoning power of large language models (LLMs) with chain-of-thought (CoT) prompting and multi-agent collaboration.

While LLMs excel at natural language understanding and problem decomposition, they struggle with execution, tool use, and adaptability in complex scenarios. LLM-Collab addresses these limitations by orchestrating two distinct agents, an analyst and an executor, who work together to analyze problems, choose external tools, and iteratively refine solutions.

Prompts are carefully done using few-shot examples made using the chain-of-thought prompting idea and instructions that guide each agent's behavior. [2]

Simulating Classroom Education with LLM-based agents

The simulating Classroom explores the potential of simulating classroom environments using LLM-based agents to better understand social learning dynamics and to develop scalable, controllable education simulations. Rather than using LLMs just as tutors or students in isolation, the authors simulate full classroom interactions with multiple autonomous agents taking on roles such as students, teachers, and observers. [17]

Scaling Large LLM Multi-Agent Collaboration

It investigates how collaboration among LLM-based agents evolves as team size increases, tasks become more complex, and model capabilities vary. The primary objective is to understand the emergent behaviors, coordination strategies, and co-operative dynamics that arise when multiple LLM-based agents are cooperating to resolve tasks that demand advanced reasoning and coordination. [9]

Chapter 3

Proposed Approach

3.1 Using LLM in the LLE

3.1.1 Exploring multi-agent collaboration with LLM

As shown in the study by `Li et al.` [5], when using LLMs in multi-agent environments, the models are not trained through a reward-based system. Instead, they rely on zero-shot or few-shot prompting.

This would be one of the approaches explored in this project to implement LLMs in the Laser Learning Environment. In contrast to methods used like VDN, this method uses direct reasoning through prompting.

Agents will operate as LLM-driven entities capable of reasoning, planning, and communicating based on structured textual prompts.

To implement this approach within LLE:

- Agents will take turns to interact with the environment when communicating with each other and taking the actions.
- For prompting framework, each agent will have their dedicated LLM session where they will have :
  - **Initial prompt**: A zero-shot prompt will be provided containing:
    - The agent identity and role
    - The game description and rules
    - Initial observation
    - Belief state format

- **Step-by-step prompting (each round)**: after the initial setup, at each round, the agent receives:
  * An update of its current observation
  * Its updated belief state, which summarizes what the agent has previously seen or inferred
  * A request to generate an action and a message

The hypothesis is that, by using structured communication and reasoning (rather than reinforcement signals), LLM agents may be able to exhibit *emergent collaborative behavior*.

Chain-of-thought Prompting

In this project, Chain-of-Thought (CoT) prompting is used as a central mechanism to enhance reasoning, planning, and cooperative behavior among LLM-based agents. Rather than producing an action directly from observation, the LLM is prompted to reason through the situation step-by-step before arriving at its decision.

Each prompt will therefore include a structured request to ”think aloud” in natural language before selecting an action. This encourages the model to articulate its interpretation of the environment, its current goal, the constraints it faces, and its plan to proceed. This serves two purposes:

- It makes the reasoning process more interpretable for both developers and teammates;
- It provides a natural structure for debugging, adjusting, or verifying agent logic.

For example, rather than issuing a direct command like:

```
Action: Move East
```

The LLM will be prompted to reason as follows:

```
[Reasoning]
 I see that the gem is located two tiles
 to the east.
 There is no laser blocking the way, and Agent B
 is not in the corridor.
 Moving east will bring me closer to the gem
 without danger.
[Action] Move East
```

This method is inspired by empirical findings showing that CoT prompting improves both task performance and robustness on complex reasoning benchmarks [14].

Additionally, in multi-agent settings, the chain-of-thought (CoT) generated by an agent can serve as a foundation for communication. When shared, this articulated reasoning can help others infer the agent's intentions, beliefs, and interpretation of the task, aligning with the goals of Theory of Mind (ToM) reasoning.

Overall, CoT prompting plays a dual role: it enhances the agent's internal reasoning and, supports external cooperation by making thought processes more interpretable to teammates.

Belief State Prompting

In this project, belief states will be implemented as structured text blocks that evolve at each round of interaction.

At each round, the agent's belief state will be regenerated to reflect:

- The agent's most recent observation (e.g., room contents, position);
- Known information about teammates (e.g., locations, tool inventory);
- Communication history from other agents;
- Environmental structure (e.g., room connectivity or bomb locations).

An example belief state from the initial prompt might look like:

```
[Belief State - Round 3]
Your role: You are Agent B (Green).
Team score so far: 2
(Agent A and Agent D have exited).
Your location: (4,2)
Observation: Ahead lies a green laser at (5,2),
which is safe to cross.
Teammate Locations: Agent C was at (3,2);
Messages received: Agent C: "an active
red laser at (5,3)"
Action: Move to (5,2)
```

This structure helps preserve continuity across rounds in an otherwise stateless LLM setup.

Theory of Mind prompting

While the belief state acts as structured memory, Theory of Mind (ToM) reasoning will actively prompted during agent interaction. Agents will be periodically required to answer questions that probe their ability to reason about the beliefs of others.

These ToM prompts test three levels of reasoning:

- **Introspection:** Can the agent articulate its own beliefs and mental state?
- **First-order ToM:** Can the agent infer what another agent knows or does not know?
- **Second-order ToM:** Can the agent reason about what one agent believes about another's knowledge?

ToM prompts are injected at key moments in the mission, typically right after an action, and framed in natural language. For example for a first-oder ToM:

```
[ToM Question]
Do you think Agent A knows that the gem
there is a laser beam further away ? Why?
```

The agent is expected to use the current belief state to reason about teammate perspectives. A valid response might include references to prior messages, teammate location, or visibility conditions:

```
No, I don't agent A where the laser beam is.
He was near the start tile.
```

These ToM checks are not performed continuously but are introduced at specific points during the mission as structured evaluation prompts. Their purpose is to assess whether agents are capable of reasoning about others' mental states inference without encoding that logic explicitly into the belief state.

By combining belief state prompting with targeted ToM evaluation, we aim to reproduce and extend the cooperative behaviors observed in prior work [5], but this time within the Laser Learning Environment (LLE).

3.1.2 Exploring multi-agent collaboration with LLM and VDN

To assess the strengths and limitations of large language models (LLMs) compared to traditional multi-agent reinforcement learning (MARL) techniques, this project may also explore a hybrid setup that combines LLM agents with a reinforcement learning baseline, specifically, Value Decomposition Networks (VDN) in the Laser Learning Environment.

VDN has been shown to perform well in the Level 6 2.1of the Laser Learning Environment, where agents must cooperate facing several challenges such as: perfect coordination, interdependence and zero-incentive dynamics. However, some difficulties remain, as no existing approach has managed to consistently reach the highest attainable score.

On the other hand, LLMs offer high-level planning and reasoning abilities through zero-shot or few-shot prompting. However, they may struggle with precise action execution and adapting to the environment in the way traditional reinforcement learning methods allow.

LLM-based agents face several challenges, including hallucinations, taking illegal actions, and limited memory. Despite these issues, their main strength lies in their strong reasoning and abstraction capabilities.

By combining both approaches, we aim to investigate whether their respective strengths can be complementary.

This mixed approach draws inspiration from the survey by Cao et al. [3], which presents a taxonomy of LLM-enhanced RL. According to the paper, LLMs can take on various roles in reinforcement learning pipelines: as *information processors*, *reward designers*, *decision-makers*, or *generators*.

Thus, within this framework, LLMs in our hybrid setup will act as decision-makers (providing plans or candidate actions) and information processors (interpreting observations or task descriptions), while VDN handles the direct policy learning and decentralized control over agent actions.

Then, if time allows, the experimental design could explore others configurations such as

- **LLM + VDN (Assisted Planning)**: LLMs act as planners or communication generators, producing natural language plans or coordination messages. These plans are passed to VDN-controlled agents, which then translate them into concrete movement or manipulation actions.

While these hybrid setups can be really interesting, putting them into practice is still a work in progress. Right now, the design remains quite abstract, and a lot of details still need to be figured out, especially around how to connect LLM-generated plans to the actual action execution handled by VDN.

3.2 Technical aspect of the implementation

3.2.1 About the LLMs models

This section explores the choice of LLMs for the project, comparing models like GPT-4 with open-source alternatives.

Using GPT-4 API model

One approach is using the GPT-4 API model, it has the advantage that it has demonstrated a powerful, state-of-the-art model that has already shown promising results in multi-agent collaboration tasks [5].

It has been proven that it supports zero-shot and few-shot reasoning, has strong language understanding, and has already been tested in belief-state and Theory of Mind (ToM) scenarios.

The main limitation of this option is cost. Every prompt sent to the model (for each agent, at each round) consumes tokens and could lead to significant API usage fees. This could become a constraint, especially when running many trials, or testing with long interaction histories.

However, given the nature of this project, which relies on zero-shot or few-shot prompting, and the fact that it does not require shaped rewards or long training phases to discover collaborative behaviors, the overall cost is likely to remain lower than traditional AI training approaches.

Using an open-source LLM locally

Another approach would be to use an open-source LLM, such as LLamA or Mistral, and run it locally. This would remove the cost constraint and provide more flexibility in experimentation, especially when designing custom memory components or modifying the way prompts are handled.

On the other hand, most open-source models are less capable than GPT-4 when it comes to complex multi-step reasoning. Some models may also have limited context windows, which is a critical factor when agents need to process history or belief states.

Running the model locally also requires significant computing power, often needing a high-performance GPU to run efficiently.

Nevertheless, beyond the technical trade-offs, it could also be interesting to compare how a different type of LLM perform on this kind of task. Testing a variety of models may reveal differences in how they reason, coordinate, or handle uncertainty even under the same kind of prompting structure. This could lead to insights on what capabilities matter most for multi-agent collaboration and where current models fall short.

When checking the existing LLM leaderboards where we get public rankings,

such as the LLM Arena[1], which is a crowdsourced battle platform, or LLM-Stat[2], which bases its leaderboard on standardized benchmarks, we can identify strong candidates.

Among the open-source models, four stand out consistently for their high performance and reasoning ability:

- DeepSeek R1
- Mistral
- LLaMA 4
- Qwen 3

In this case, DeepSeek emerges as the most suitable choice, as it consistently outperforms the others, making it a good candidate for our needs in multi-agent collaboration.

Selected Implementation Strategy

Based on the considerations discussed above, the selected approach is to go whith an open-source language model and run the model locally. Since the number of interactions or episodes needed to evaluate and compare strategies is still unknown, this approach avoids API cost limitations and gives more freedom during experimentation. It also makes it easier to scale up if needed.

Another reason is that it would be interesting to test with a different LLM than the one used in the paper by Li et al [5]. Using another model allows us to see whether the same prompting framework and belief-state setup can work beyond GPT-4. This could help assess how transferable and general the method is.

Running locally also gives us more control and flexibility. This makes it easier to adapt the setup to the needs of the task, and to explore variations without being limited by a fixed API.

In terms of model choice, we will begin with DeepSeek, as it consistently ranks among the top open-source models on several public benchmarks.

If time allows, it could also be interesting to test how different model sizes or architectures affect collaboration quality, for example, whether smaller LLMs can still perform well with the right belief state prompts. It could also be interesting to compare other open-source models like Mistral, Qwen, or LLaMA under the same setup.

[1] `https://lmarena.ai/leaderboard/text`
[2] `https://llm-stats.com/`

3.2.2 Integrating LLM-based Agents in the LLE Environment

The implementation of LLM-based agents will be done by extending the existing LLE environment, rather than adapting the LLE environment to fit the external codebase used in the paper of `Li et al.` [5]. This choice ensures full compatibility with the original LLE dynamics, observation formats, and reward signals, and avoids unnecessary abstraction layers.

Design Overview

The integration will follow these principles:

- The LLE environment, will remain unchanged in terms of environment logic, level structure, and agent interface.
- A new wrapper class will be created to implement a `LLMAgent` that communicates with an external LLM process or local model (e.g., DeepSeek).
- The LLM-based agent will operate in turn-based fashion, using the existing LLE step function to interact with the environment.
- All observations, belief states, messages, and actions will be handled in natural language and parsed using a dedicated text interface module.

3.2.3 Results Evaluation

To evaluate the performance of the LLM-based agents in the Laser Laser Environment, several metrics will be used. Among them, we will use the same metrics proposed in the original LLE paper [6]:

- **Score** which refers to the total number of points collected during an episode, without applying any discount. The maximum possible score in LLE depends on the map: if there are n agents and g gems, the best possible score is $n + g + 1$ (with the extra +1 coming from the team bonus when all agents reach the exit).
- **Exit rate** that measures how many agents make it to the exit by the end of the episode. It gives a good idea of how close the team gets to completing the task. An exit rate of **1** means that all agents successfully reached the goal.

These results will then be compared to the baseline performance of VDN on Level 6 of LLE [6]. Since VDN is considered the best-performing MARL baseline in that paper, it gives us a good reference to see if LLM-based agents can reach similar or better results.

In addition, we will include two qualitative metrics to assess reasoning quality and prompting strategy:

- **Theory of Mind reasoning**: whether agents show signs of Theory of Mind (e.g., adapting their behavior based on what others know or don't know).
- **Chain-of-Thought vs. Standard Prompting**: whether prompting the LLMs to "think aloud" using structured reasoning steps improves decision quality, coordination, and success rate compared to directly outputting actions.

To evaluate reasoning, we will rely on manual analysis of selected episodes, focusing on situations where agents are expected to reason about the mental states of others. This includes identifying moments where an agent acts (or chooses not to act) based on what it believes another agent knows, has seen, or has been told.

The evaluation will follow a few simple criteria inspired by the setup in the Li et al. paper [5]:

1. *Did the agent demonstrate awareness of what another agent has or has not observed (e.g., based on location or visibility)?*
2. *Did the agent respond appropriately to information received through communication?*
3. *Did the agent act in a way that suggests second-order reasoning (e.g., "they don't know that I know X")?*

By reviewing the interaction history, we will be able to assess whether agents are applying some form of Theory of Mind reasoning in practice.

Similarly, to assess the effect of Chain-of-Thought prompting, we will compare agent behavior under two configurations:

- **Standard prompting**, where the model is asked to output an action directly from its observation.
- **Chain-of-Thought prompting**, where the model is encouraged to reason through the situation step-by-step before selecting its action.

This comparison will allow us to evaluate whether explicit reasoning improves collaboration and decision quality. Metrics such as exit rate, coherence of generated plans, and response alignment will be analyzed to determine the contribution of CoT prompting to agent performance.

Finally, to better understand the impact of belief states, we will compare agent performance in two settings: one where belief states are included and updated at each step, and another where agents rely only on their current observations and last received messages. This will help assess whether belief-state prompting enhances coordination, adaptability, and continuity of reasoning across steps.

Chapter 4

Conclusion

This project investigates the integration of large language models (LLMs) into multi-agent collaboration within the Laser Learning Environment (LLE), aiming to go beyond traditional reinforcement learning (RL) methods. While prior work in LLE has shown that algorithms like Value Decomposition Networks (VDN) can perform well in high-dependency tasks, they often struggle with complex cooperation that requires implicit reasoning and long-term planning without immediate rewards.

Inspired by recent advances in Theory of Mind (ToM) reasoning, this work proposes a new framework where LLM-based agents reason, plan, and communicate through structured natural language. These agents use belief states to simulate memory and awareness of teammates' mental states, enabling more human-like coordination.

By using zero-shot or few-shot prompting, these agents are not trained inside the environment, but rather guided through careful design of their interaction pipeline: including belief state updates, CoT reasoning, and structured message passing.

The proposed architecture will be evaluated against VDN baselines, using both quantitative metrics (score, exit rate) and qualitative observations of ToM-style reasoning. A particular focus is placed on Level 6 of LLE, which includes complex interdependence, hidden information, and the need for communication.

Bibliography

[1] Nolan Bard, Jakob N Foerster, Sarath Chandar, Neil Burch, Marc Lanctot, H Francis Song, Emilio Parisotto, Vincent Dumoulin, Subhodeep Moitra, Edward Hughes, et al. "The hanabi challenge: A new frontier for ai research". In: *Artificial Intelligence* 280 (2020), p. 103216.

[2] Hong Cao, Rong Ma, Yanlong Zhai, and Jun Shen. "LLM-Collab: a framework for enhancing task planning via chain-of-thought and multi-agent collaboration". In: *Applied Computing and Intelligence* 4 (Jan. 2024), pp. 328–348. DOI: 10.3934/aci.2024019.

[3] Yuji Cao, Huan Zhao, Yuheng Cheng, Ting Shu, Yue Chen, Guolong Liu, Gaoqi Liang, Junhua Zhao, Jinyue Yan, and Yun Li. "Survey on Large Language Model-Enhanced Reinforcement Learning: Concept, Taxonomy, and Methods". In: *IEEE Transactions on Neural Networks and Learning Systems* 36.6 (2025), pp. 9737–9757. DOI: 10.1109/TNNLS.2024.3497992.

[5] Huao Li, Yu Chong, Simon Stepputtis, Joseph Campbell, Dana Hughes, Charles Lewis, and Katia Sycara. "Theory of Mind for Multi-Agent Collaboration via Large Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 180–192. DOI: 10.18653/v1/2023.emnlp-main.13.

[6] Yannick Molinghen, Raphaël Avalos, Mark Van Achter, Ann Nowé, and Tom Lenaerts. "Laser Learning Environment: A new environment for coordination-critical multi-agent tasks". In: *BNAIC 2023*. BeNeLux Artificial Intelligence Conference. 2023.

[9] Chen Qian, Zihao Xie, YiFei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. *Scaling Large Language Model-based Multi-Agent Collaboration*. 2025. arXiv: 2406.07155 [cs.AI]. URL: https://arxiv.org/abs/2406.07155.

[10] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. "Monotonic value function

factorisation for deep multi-agent reinforcement learning". In: *Journal of Machine Learning Research* 21.178 (2020), pp. 1–51.

[13] Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. "Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward". In: *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*. AAMAS '18. Stockholm, Sweden: International Foundation for Autonomous Agents and Multiagent Systems, 2018, pp. 2085–2087.

[14] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[15] Sarah A Wu, Rose E Wang, James A Evans, Joshua B Tenenbaum, David C Parkes, and Max Kleiman-Weiner. "Too many cooks: Coordinating multi-agent collaboration through inverse planning". In: *Proceedings of the annual meeting of the cognitive science society*. Vol. 42. 2020.

[17] Zheyuan Zhang, Daniel Zhang-Li, Jifan Yu, Linlu Gong, Jinchang Zhou, Zhanxin Hao, Jianxiao Jiang, Jie Cao, Huiqin Liu, Zhiyuan Liu, Lei Hou, and Juanzi Li. "Simulating Classroom Education with LLM-Empowered Agents". In: (2024). arXiv: 2406.19226 [cs.CL]. URL: https://arxiv.org/abs/2406.19226.

Complementary References

[4] Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L Griffiths, and Mengdi Wang. "Embodied LLM Agents Learn to Cooperate in Organized Teams". In: *arXiv preprint arXiv:2403.12482* (2024).

[7] Ini Oguntola, Joseph Campbell, Simon Stepputtis, and Katia Sycara. *Theory of Mind as Intrinsic Motivation for Multi-Agent Reinforcement Learning*. July 2023. DOI: 10.48550/arXiv.2307.01158.

[8] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. "Generative Agents: Interactive Simulacra of Human Behavior". In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. UIST '23. San Francisco, CA, USA: Association for Computing Machinery, 2023. ISBN: 9798400701320. DOI: 10.1145/3586183.3606763.

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. "Reflexion: Language agents with verbal reinforcement learning". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 8634–8652.

[12] Chuanneng Sun, Songjun Huang, and Dario Pompili. *LLM-based Multi-Agent Reinforcement Learning: Current and Future Directions*. 2024. arXiv: 2405.11106 [cs.MA].

[16] Hui Yang, Sifu Yue, and Yunzhong He. *Auto-GPT for Online Decision Making: Benchmarks and Additional Opinions*. 2023. arXiv: 2306.02224 [cs.AI].